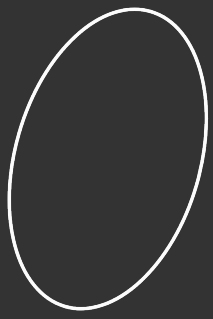
# Observer Design Pattern.

# ⊛ Observer Design Pattern

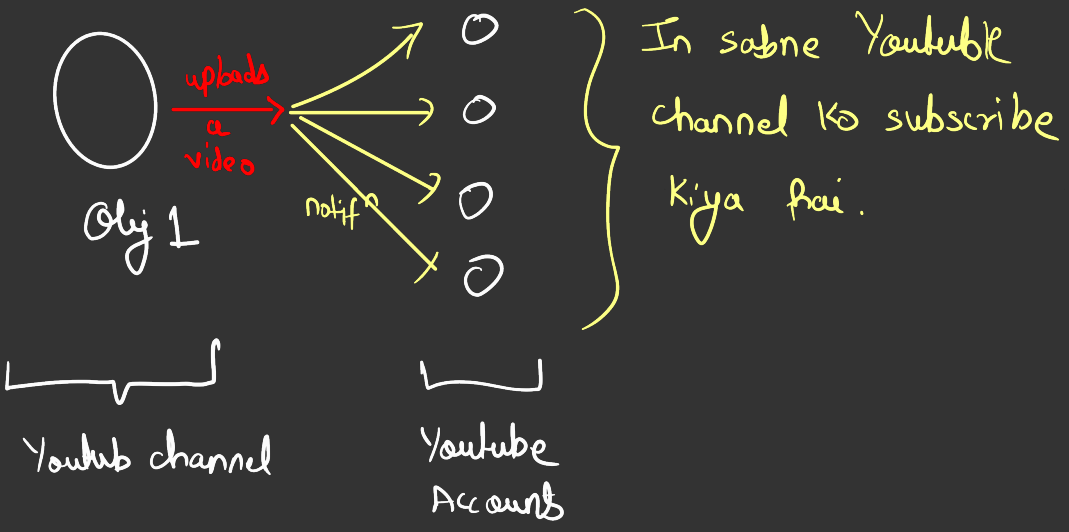
→ The observer design pattern is a behavioral design pattern that defines a one-to-many dependency between objects. When one Object (the subject) changes state, all its dependents (observers) are notified and updated automatically. This pattern is commonly used in event handling, GUI programming & situations where real-time updates are needed.

→ Observer pattern me humare pass ek object hota.



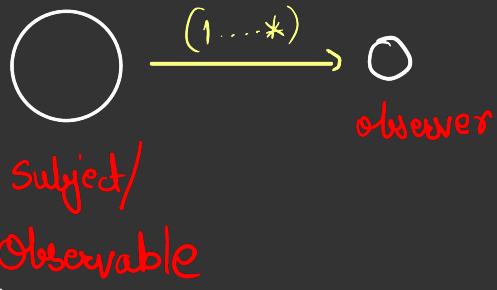
Obj 1

→ Ab aur objects hai jo janana chahte hai Ki is Obj 1 Ki internal state Kab change hoti hai.



# ⊛ Polling Technique

→ Let's take same eg.




---

Polling me, observer jake puchega Observable se ki tumhari value change hui hai ki nahi ?

⊙ Kab puchega ?

→ Periodically

Very Time Consuming, lots of requests needs to be made by Observer. Not a good approach.

We should move from Polling to Pushing

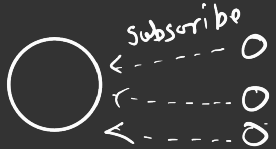
# Pushing Technique

→ It should be responsibility of Observable to notify observers whenever its state changes


→ Observable should be aware of observers so that it knows whom to send notifications.

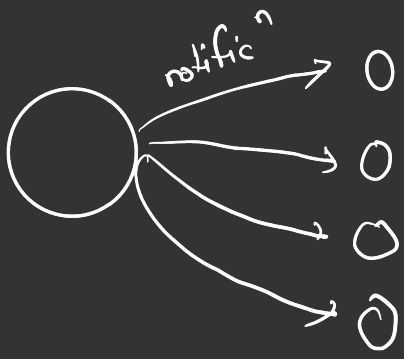
→ Observable will internally use data-structure like list to store observer's Info.

**Step1** :→ Observers will subscribe to Observable.




with this observable will be aware of subscribed observers.

Step 2 :→ Jab bhi Observable ki internal state change hogi [YT video upload in our case], so Observable will send notific^n to all observers.




This is what is Observer Design Pattern

---

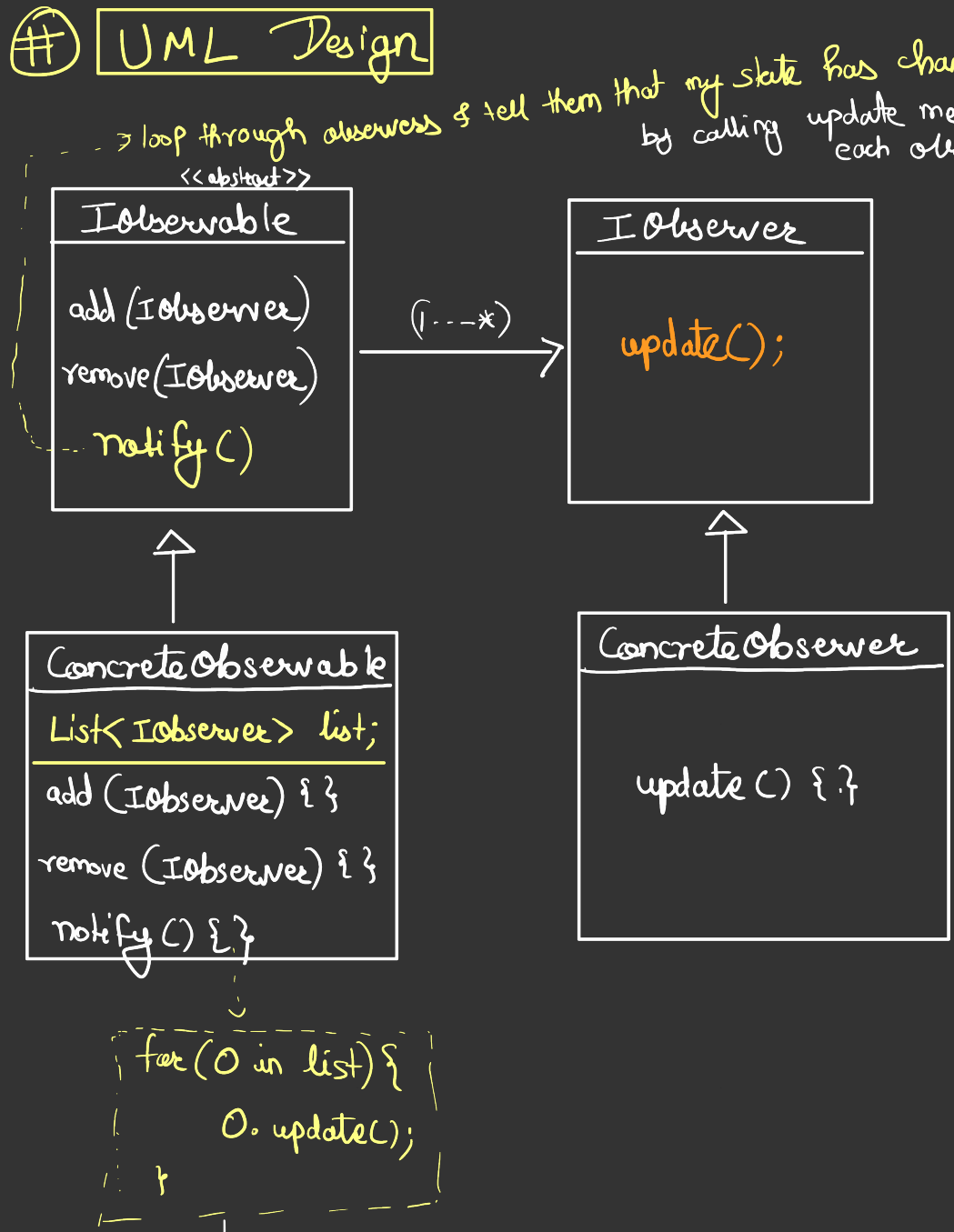
Note :→ From now on, we will have naming convent^n for Interfaces

Eg :→ IChannel, ICar ....

↓

This means all methods present here needs to be implemented by child.

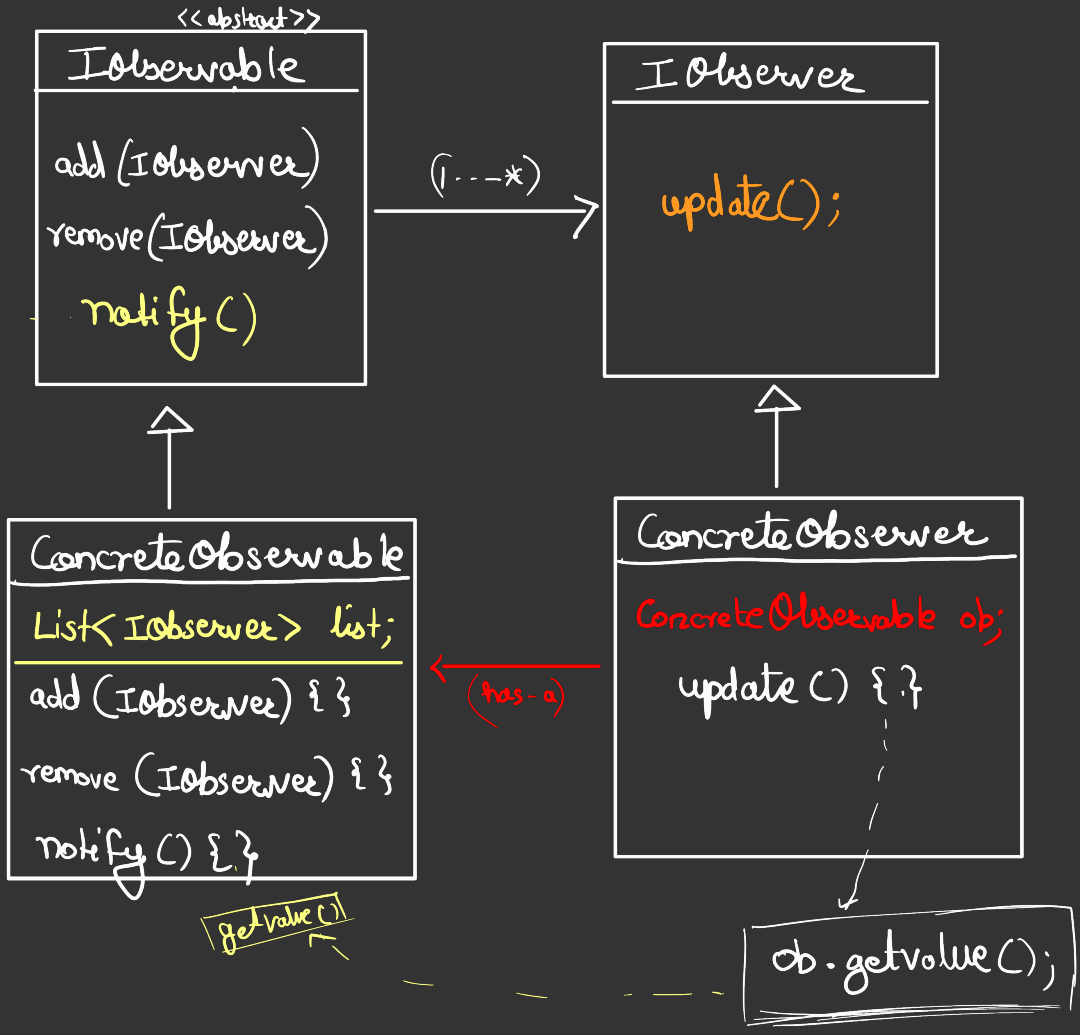
# UML Design



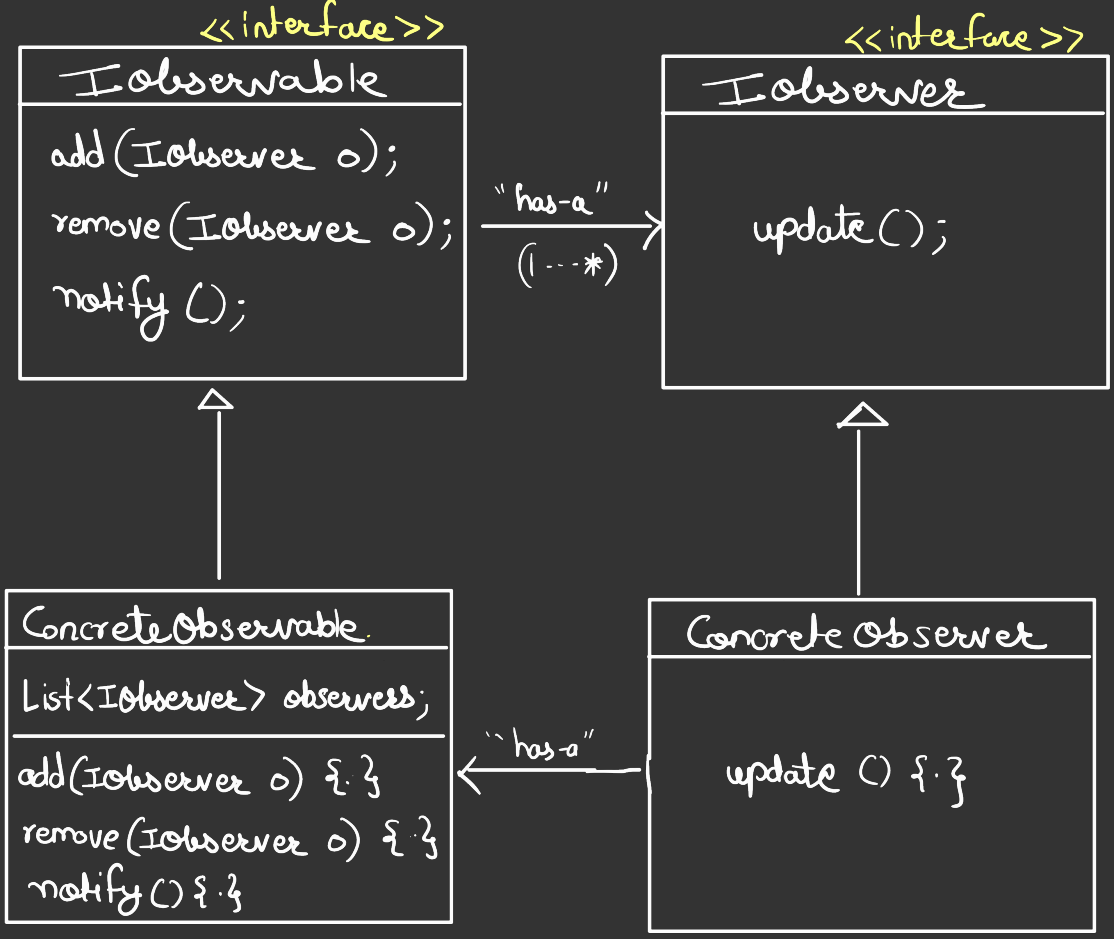

↳ OK, O will receive notification but how will it understand from which Observable this came? It might have subscribed multiple

# Continue

Hum ConcreteObserver me ConcreteObservable bhi pass kar dete hai.



# Standard UML

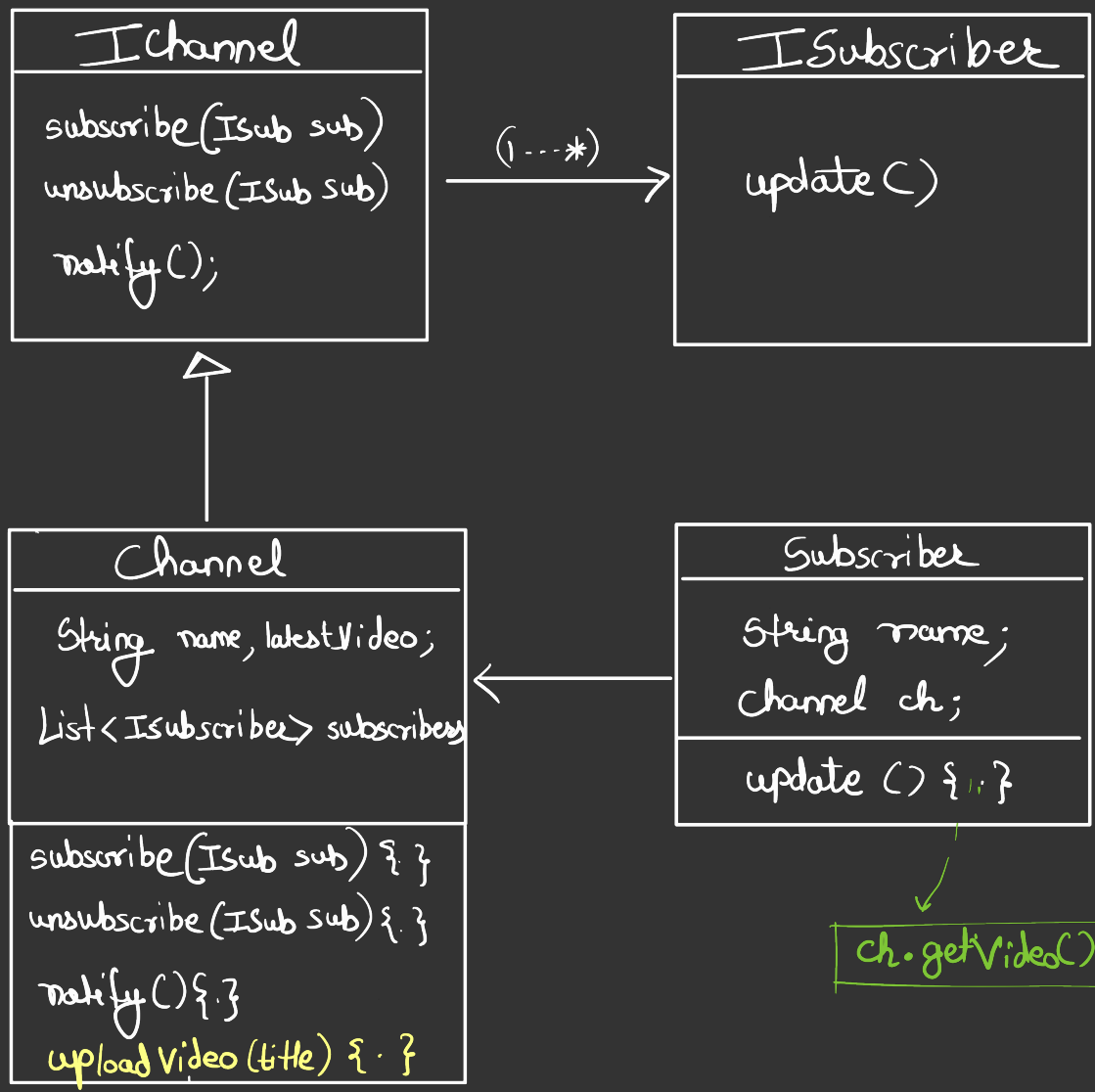



---

**Defn :→**

Define a one to many relationship between objects so that when object changes state, all its dependent are notified & updated automatically.

# Youtube Example



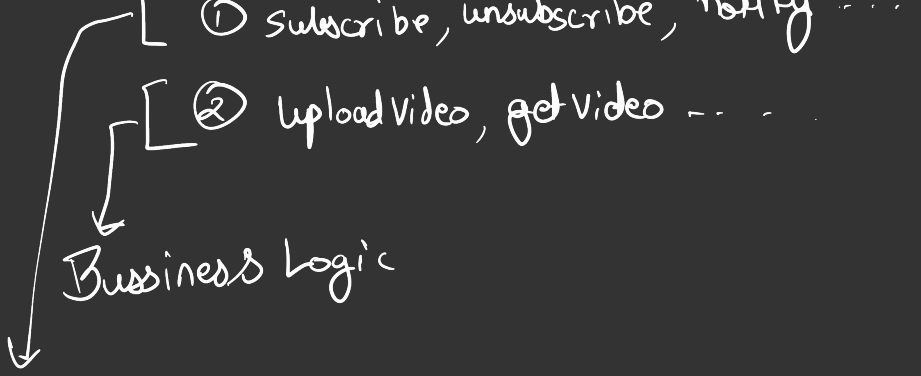

Jab bhi uploadvideo call hota hai, vo internally notify ko call karta hai. Notify will then notify the message to subscribers.

# Few Observations

→ It breaks one SOLID Design Principle "SRP"

## Concrete Channel

↳ Ye do kam kar raha hai.




Observer Design Pattern code

---

## It's a trade-off

If you don't want to break SRP then subscribe, unsubscribe and notify method ko IChannel abstract class me define karo. Now IChannel is abstract class not interface. Aur in methods me concrete class wali subscribers list bhej

# Real-World Examples

1. Notification Service.
2. NewsFeed Service. [Social Media].
3. Event handling.